

实验九：C++的输入输出流库

一、实验目的

1. 学习进行格式化输入输出。
2. 掌握文件的输入输出操作。

二、实验要求

1. 输出十进制、八进制、十六进制显示的数据 0~15。
2. 类 `stu` 用来描述学生的姓名、学号、数学成绩、英语成绩，分别建立文本文件和二进制文件，将若干学生的信息保存在文件中，读出该文件的内容。
3. 设计一个留言类，实现以下的功能。
 - (1) 程序第一次运行时，建立一个 `message.txt` 文本文件，并把用户输入的信息存入该文件。
 - (2) 以后每次运行时，都先读取该文件的内容并显示给用户，然后由用户输入新的信息，退出时将新的信息存入这个文档。文件的内容，既可以是最新的信息，也可以包括以前所有的信息，用户可自己选择。

三、实验原理

1. C++ 流的概念

C++ 将所有输入输出操作抽象为字节流（stream）：

输入流：数据从文件 / 键盘流入内存

输出流：数据从内存流出到文件 / 屏幕

流操作屏蔽了底层设备差异，使文件读写与控制台输入输出语法一致。

2. 流库头文件

<iostream>提供控制台流：

`cin`：标准输入

`cout`：标准输出

<fstream>提供文件流类，是本次实验核心：

`ifstream`：文件输入流（读文件）

`ofstream`: 文件输出流（写文件）
<string>用于存储留言内容字符串。

3. 文件操作基本原理

(1) 文件打开与创建

使用 `ofstream` 打开文件时:

文件不存在 → 自动创建新文件

文件已存在 → 根据打开模式决定覆盖或追加

这实现了题目要求: 第一次运行自动建立 `message.txt`。

(2) 文件读取原理 (`ifstream`)

通过 `getline`(流对象, 字符串) 逐行读取文本内容。

若文件打开失败 (`!ifs.is_open()`), 说明是第一次运行, 无历史留言。

(3) 文件写入原理 (`ofstream`)

使用 流插入运算符 `<<` 将字符串写入文件。

支持两种模式:

`ios::trunc`: 清空原文件内容, 只保存最新留言 (覆盖)

`ios::app`: 在文件末尾添加新内容 (保留全部历史)

4. 程序运行流程原理

启动 → 以读方式打开 `message.txt`

能打开 → 读取并显示历史留言

不能打开 → 判定为首次运行, 文件不存在

用户输入新留言

用户选择保存模式: 覆盖 / 追加

以对应模式写入文件, 完成持久化存储

5. 面向对象结合原理

设计 `Message` 类封装留言内容与文件操作。

类内提供:

读文件函数 `readFromFile()`

写文件函数 `saveToFile()`

实现数据与操作的封装, 符合面向对象程序设计思想。

四、实验内容与步骤

实验内容:

1. 第一题

【实验步骤】

Step 1: 创建项目

创建名为 shuchu 的 C++ 项目。

Step2: 代码编写

shuchu.cpp 文件

```
#include <iostream>    // 包含 C++ 标准输入输出流库
#include <iomanip>      // 包含格式控制头文件（用于进制、对齐）
using namespace std;   // 方便使用标准库，适合初学者

int main() {
    // 输出表头，让结果更清晰
    cout << "数字\t十进制\t八进制\t十六进制" << endl;
    cout << "-----" << endl;

    // 循环遍历 0~15 的所有数字
    for (int i = 0; i <= 15; i++) {
        // 输出当前数字

        // 十进制输出（dec 是 C++ 默认进制，可写可不写）

        // 八进制输出（oct 关键字）

        // 十六进制输出（hex 关键字）

    }

    return 0;
}
```

2. 第二题

Step 1: 创建项目

创建名为 Student 的 C++ 项目。

Step2: 代码编写

Student.cpp 文件

```
// 填空 1: 补充所需头文件
#include <iostream>
#include _____ // 文件流头文件
```

```

#include _____ // string 字符串头文件
using namespace std;

// 定义学生类 stu
class stu {
private:
    string name;    //姓名
    int id;         //学号
    double math;    //数学
    double english; //英语
public:
    //给成员变量赋值
    void setInfo(string n, int i, double m, double e) {
        name = n;
        id = i;
        math = m;
        english = e;
    }
    void showInfo() {
        cout << name << " " << id << " " << math << " " << english << endl;
    }

    //填空 2: 声明友元, 重载文本输出运算符<<
    friend _____ operator<<(___, stu& s);
    //填空 3: 声明友元, 重载文本输入运算符>>
    friend _____ operator>>(___, stu& s);

    //二进制写入函数
    void writeBinary(ofstream& ofs) {
        //填空 4: 二进制整块写入对象
        ofs.write(_____, _____);
    }
    //二进制读取函数
    void readBinary(ifstream& ifs) {
        //填空 5: 二进制整块读取对象
        ifs.read(_____, _____);
    }
};

//重载<<实现: 文本写入
ofstream& operator<<(ofstream& ofs, stu& s) {
    ofs << s.name << " " << s.id << " " << s.math << " " << s.english;
    return ofs;
}

```

```

//重载>>实现：文本读取
ifstream& operator>>(ifstream& ifs, stu& s) {
    ifs >> s.name >> s.id >> s.math >> s.english;
    return ifs;
}

int main() {
    stu s1, s2;
    s1.setInfo("张三", 1001, 90.5, 85);
    s2.setInfo("李四", 1002, 88, 92.5);

    //====文本文件操作====
    //填空 6：定义文本输出流对象，打开 student_text.txt
    _____ outText("student_text.txt");
    outText << s1 << s2;
    outText.close();

    //填空 7：定义文本输入流对象
    _____ inText("student_text.txt");
    stu tmp;
    cout << "文本读出：" << endl;
    while (inText >> tmp) {
        tmp.showInfo();
    }
    inText.close();

    //====二进制文件操作====
    //填空 8：二进制方式打开文件，补充打开模式
    ofstream outBin("student.dat", _____);
    s1.writeBinary(outBin);
    s2.writeBinary(outBin);
    outBin.close();

    //填空 9：二进制读文件
    ifstream inBin("student.dat", _____);
    stu t;
    cout << "二进制读出：" << endl;
    while (inBin.read((char*)&t, sizeof(stu))) {
        t.showInfo();
    }
    inBin.close();
    return 0;
}

```

3 第三题

【实验步骤】

Step 1: 创建项目

创建名为 Message 的 C++ 项目。

Step2: 代码编写

Message.cpp 文件

```
#include <iostream>
#include <fstream>
#include <string>
#include _____ //字符串流头文件
using namespace std;

class Message
{
private:
    string msgContent;
public:
    void setMsg(string s)
    {
        _____; //给成员赋值
    }
    string getMsg(_____ ) //常成员函数
    {
        return msgContent;
    }

    //读取文件
    string readFromFile(const string& filename)
    {
        _____ ifs(filename); //定义文件读对象
        string all, line;
        if (!ifs.is_open())
            return "";
        while (_____ ) //循环逐行读取
        {
            all += line + "\n";
        }
        ifs.close();
        return all;
    }
}
```

```

    }

    //mode:1 覆盖 2 追加
    bool saveToFile(const string& filename, const string& newData, int mode)
    {
        ofstream ofs;
        if (mode == 1)
            ofs.open(filename, _____); //覆盖清空标志
        else if (mode == 2)
            ofs.open(filename, _____); //追加标志

        if (!ofs.is_open())
            return false;
        _____; //把新数据写入文件
        ofs.close();
        return true;
    }
};

int main()
{
    Message m;
    string fname = "message.txt";
    string history = _____; //调用读取函数

    cout << "历史留言: \n" << history << endl;
    string newMsg;
    cout << "输入新留言: ";
    getline(cin, newMsg);
    _____; //调用 setMsg 存新留言

    int opt;
    cout << "1 覆盖 2 追加:";
    cin >> opt;
    if (opt == 1 || opt == 2)
        _____; //调用保存函数

    return 0;
}

```

实验提交要求

正确编译、运行并截图结果。

记录实验中遇到的错误及解决方法。

提交关键源代码、运行截图、实验报告

答案页

第一题

运行结果示例：

数字	十进制	八进制	十六进制
0	0	0	0
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	5	5	5
6	6	6	6
7	7	7	7
8	8	10	8
9	9	11	9
A	10	12	A
B	11	13	B
C	12	14	C
D	13	15	D
E	14	16	E
F	15	17	F

第二题

运行结果示例：

```
===== 文本文件写入与读取 =====
文本文件写入成功！
文本文件内容：
姓名：张三      学号：1001      数学：90.5      英语：85
姓名：李四      学号：1002      数学：88       英语：92.5

===== 二进制文件写入与读取 =====
二进制文件写入成功！
二进制文件内容：
姓名：张三      学号：1001      数学：90.5      英语：85
姓名：李四      学号：1002      数学：88       英语：92.5
```

项目所在的文件夹中

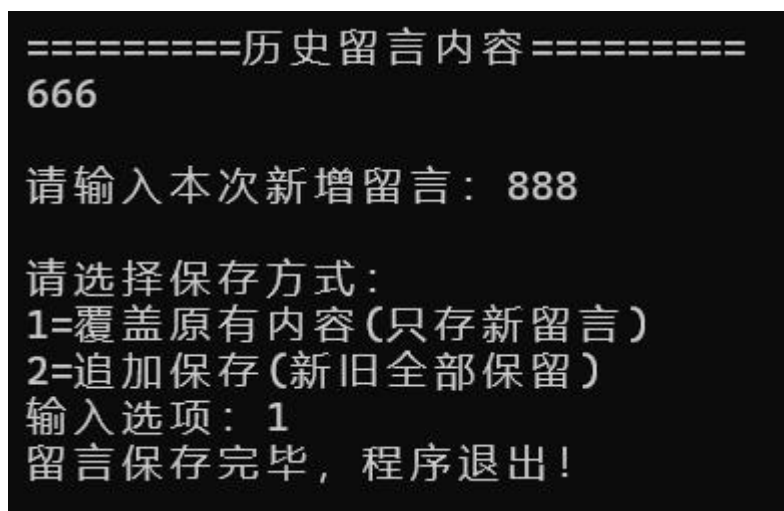
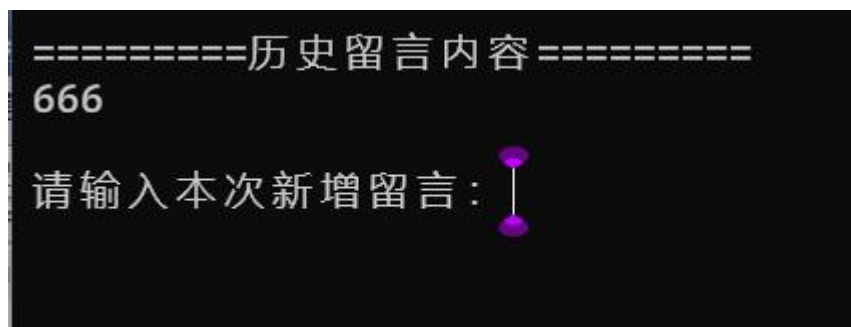
 student_binary.dat	2026/6/6 18:25	DAT 文件	1 KB
 student_text.txt	2026/6/6 18:25	文本文档	1 KB

用记事本查看 txt 文件



第三题

运行结果示例：



再次运行

